تشريح الـ Express من جوه (الـ Under the Hood)**

طب الـ Express من جوه كـ "كود" شغال إزاي؟ هو مش سحر، هو ببساطة **"مصفوفة" (Array) كبيرة جداً اسمها الـ Router Stack**.

1. **الـ `app` مجرد دالة (Function):** لما بتكتب `const app = express()` ، إنت بتعمل دالة ضخمة. الدالة دي بتتبعت لـ `http.createServer(app)` بتاع Node.js الأساسي. يعني الـ Express كله "تغليفة" شيك لموديول الـ `http` .
2. **مصفوفة الطبقات (The Stack):** كل ما بتكتب `app.use()` أو `app.get()` ، الـ Express بياخد الدالة بتاعتك دي ويغلفها في حاجة اسمها `Layer` ويرميها جوه مصفوفة.
3. **لما الطلب يوصل (The Cycle):**
   - الـ Express بيمسك الطلب ويطابق الرابط (URL) مع المصفوفة دي **من فوق لتحت بالترتيب**.
   - كل ما يلاقي طبقة (Layer) مطابقة، ينفذ الدالة بتاعتها.
   - لو الدالة نادت على `next()` ، الـ Express يدور على الطبقة اللي بعدها، وهكذا.

أنواع "العمال" (أنواع الـ Middleware)

العمال دول مش كلهم زي بعض، الـ Express مقسمهم لـ 5 أنواع:

- **عمال على باب المصنع (Application-level):** دول مربوطين بـ `app.use()` وبيشتغلوا على أي طلب يجي. (زي الـ `cors` أو الـ `express.json` اللي بيفك شفرة البيانات).
- **عمال في الأقسام (Router-level):** دول مربوطين بـ `express.Router()` . بتعملهم عشان تنظم الكود (قسم للمنتجات، قسم لليوزرات).
- **عمال مدمجين (Built-in):** دول جايين جاهزين مع الـ Express من غير ما تسطب حاجة (زي `express.static` عشان تخدم ملفات الصور والـ CSS).
- **عمال من بره (Third-party):** بتجيبهم من الـ npm (زي `morgan` عشان يسجل اللوجات، أو `helmet` عشان الحماية).
- **عامل المطافي (Error-handling):** ده الوحيد اللي شكله مختلف، بياخد 4 باراميترات `(err, req, res, next)` . ده دايماً بنحطه **في آخر الكود خالص**. لو أي عامل في النص لقى مشكلة، بيباصي المشكلة لـ `next(err)` ، والسير بيسيب كل العمال ويجري علطول لعامل المطافي ده عشان يرد بـ رسالة خطأ محترمة بدل ما السيرفر يقع.

```
app.use("/", (req, res, next) => {
  console.log("first middleware");
  next();
});

app.use("/add-product", (req, res, next) => {
  console.log("add product middleware");
```

```
  res.send("<h1>add product page</h1>");
});

app.use("/", (req, res, next) => {
  console.log("home page middleware");
  res.send("<h1>home page</h1>");
});
```

1. رحلة الطلب جوه الكود

الـ Express بيمشي **من فوق لتحت بالترتيب**، ومابيرجعش لورا. تعال نشوف السيناريوهات:

- **لو الزبون طلب `add-product/` :**
  1. هيخبط في أول كمين `app.use("/")` . الـ Express بيسأل: "هل الرابط بيبدأ بـ `/` ؟" الإجابة أيوة. هيطبع `first middleware` ، والـ `next()` هتقوله "عدي للي بعده".
  2. هيوصل للكمين التاني `app.use("/add-product")` . بيبدأ بنفس الكلمة؟ أيوة. هيطبع `add product middleware` ، ويرد على الزبون بـ `res.send()` .
  3. **هنا الرحلة بتنتهي.** الكمين التالت مستحيل يشتغل لأن الـ `res.send` قفلت الخط وسلمت البضاعة للعميل.
- **لو الزبون طلب `/` (الرئيسية) أو أي حاجة تانية زي `about/` :**
  1. هيخبط في الأول، يطبع `first middleware` ويعدي بالـ `next()` .
  2. يروح للتاني.. مش بيبدأ بـ `add-product/` ، فهيتجاهله تماماً ويسيبه ينزل للي تحته.
  3. يوصل للتالت `app.use("/")` . هيطبع `home page middleware` ويرد بـ `res.send()` .

---

2. المقلب بتاع الـ Prefix Matching (تطابق البدايات)

حتة صايعة لازم تاخد بالك منها: دالة `app.use` بتتعامل مع المسار كـ **بداية (Prefix)** مش تطابق كامل (Exact Match). يعني `app.use("/")` معناها: "أي رابط في الدنيا بيبدأ بـ `/` نفذ عليه الكلام ده". وعشان كل الروابط بتبدأ بـ `/` ، فالكمين الأول ده هيشتغل مع **أي** طلب ييجي للسيرفر.

---

3. سحر الـ `res.send()` من تحت الحوض (الهيدرز)

في الـ Node.js "الخام"، عشان ترد على الزبون بصفحة HTML، كنت لازم تكتب كذا سطر عشان تضبط الهيدرز (Headers) بإيدك بالـ `res.setHeader` وبعدين تقفل بـ `res.end()` .

الـ Express بقى جاب لك صنايعي ناصح اسمه `res.send()` بيعمل الحوارات دي كلها **أوتوماتيك** في الخلفية:

- **نوع المحتوى (Content-Type):** الـ `res.send` بتبص على الحاجة اللي إنت باعتها (زي المحقق). لقتك باعت `<h1>...</h1>` (String) ، هتقوم ضاربة في الهيدر أوتوماتيك `Content-Type: text/html` . لو كنت باعت مصفوفة أو كائن `{}` ، كانت هتحط `application/json` .

- **طول المحتوى (Content-Length):** الـ Express بيحسب وزن الرسالة بالبايت (Bytes) ويحط الهيدر ده، عشان المتصفح بتاع الزبون يبقى عارف هو هيستلم داتا قد إيه بالظبط وميستناش على الفاضي.
- **بصمة الكاش (ETag):** دي بقى حتة تقيلة.. الـ Express بيعمل "تشفير خفيف" للرد ده ويطلع له بصمة (مثلاً `W/"15-xyz..."`) ويبعتها في الهيدرز. لو الزبون عمل "ريفريش" للصفحة، المتصفح بيبعت البصمة دي للسيرفر.. السيرفر بيشوف إن الصفحة ماتغيرتش، فيرد عليه بكود `Not Modified 304` من غير ما يبعت الصفحة تاني، وده بيوفر باقة النت وسرعة السيرفر جداً!

---

1. إيه هو الـ `body-parser` ده؟ (المفتش)

لما الزبون بيبعت بيانات (زي إيميل وباسوورد من صفحة تسجيل)، البيانات دي مابتجيش حتة واحدة، بتيجي متقطعة في حاجة اسمها **Streams (تيارات)**. الـ Express بيكسل يجمعها.. فهنا بيجي دور الـ `body-parser`. ده "المفتش" اللي بيقف يجمع الحتت دي، يلزقها في بعض، ويجهزهالك على صينية فضة اسمها `req.body` عشان تستخدمها علطول في الرابط بتاعك.

2. يعني إيه `urlencoded`؟ (شفرة الـ HTML)

لما الزبون بيدوس Submit في أي `<form>` عادي في الـ HTML، المتصفح بياخد البيانات ويلزقها في بعضها كدة: `name=ahmed&age=25`. (الصيغة دي اسمها `application/x-www-form-urlencoded`). دالة `urlencoded` دي بقى هي "القاموس" اللي بيفك الشفرة دي ويحولها لـ JavaScript Object محترم (يعني `{ name: "ahmed", age: 25 }`).

3. اللغز بتاع `{ extended: false }` (سادة ولا بالمكسرات؟)

السطر ده هو "الزتونة" اللي بتعمل مشاكل كتير لو مش فاهمها:

- **`extended: false` (النسخة السادة):** اسمها Node يستخدم مكتبة أساسية جوه الـ Parser بيخلي الـ `querystring`. مابتفهمش لو باعت متداخلة جوه بعضها في Objects دي بتفهم البيانات المباشرة بس (مفتاح = قيمة). الفورم.
- **`extended: true` (بالمكسرات):** دي بقى "وحش" بتفهم لو باعت بيانات معقدة. `qs` بيستخدم مكتبة خارجية أتقل اسمها (لأن شغل الفورمز بيبقى بسيط `false` في الغالب إحنا بنستخدم). جداً ومصفوفات متداخلة من الفورم.

4. إيه اللي بيحصل في الكواليس؟ (The Flow)

1. الطلب بيوصل، بيخبط في كمين `app.use`.
2. الـ `body-parser` بيشم الـ Headers بتاعة الطلب.. "هل الـ `Content-Type` بتاعك `urlencoded`؟"
3. **لو أيوة:** يوقف سير المصنع، يجمع البيانات، يفك شفرتها، يرميها في `req.body`، وبعدين يدوس `next()` عشان يبعتك للرابط اللي عليه الدور.
4. **لو لأ:** (مثلاً الزبون باعت بيانات JSON أو صورة)، المفتش ده بيكبر دماغه، مش بيلمس حاجة، وبيدوس `next()` علطول.

---

1. الـ Status Code (الرقم الكودي)

ده "الرقم" الرسمي اللي المتصفح أو الـ API بيفهمه. هو عبارة عن 3 أرقام بتلخص حالة الطلب بالمللي.

- **وظيفته:** بيعرف المكنة (Browser, Postman, Mobile App) إيه اللي حصل من غير ما يقرأ رغي كتير.
- **أمثلة شهيرة:**
  - **200:** الدنيا تمام (OK).
  - **201:** مبروك، الداتا اتكريتت (Created).
  - **404:** اللي بتدوره عليه مش هنا (Not Found).
  - **500:** السيرفر "جاب جوانب" وفيه غلطة كود (Internal Server Error).

2. الـ Status (الحالة / الرسالة)

في الـ HTTP، ده بيبقى اسمه **"Reason Phrase"** أو رسالة الحالة. دي بتبقى "الكلمة" اللي بتيجي جنب الرقم عشان إحنا كبشر نفهمها.

- **وظيفتها:** توضيح بشري للرقم الكودي.
- **مثال:** لما السيرفر يبعت `Not Found 404` .. الـ **404** هي الـ Status Code، والـ **Not Found** هي الـ Status.

---

3. الفرق في الـ Express.js (مهم جداً)

لو إنت بتكتب كود Express، هتلاقي عندك دالتين شكلهم يلخبط:

أ- `res.status(code)`

دي دالة وظيفتها بس إنها "تظبط" الرقم في الـ Header بتاع الرد. هي مش بتبعت الرد للعميل، هي بس بتحضر "الرتبة".

JavaScript

```
res.status(404); // كدة إنت جهزت إن الرد هيبقى 404، بس لسه مابعتش حاجة.
res.send("صفحة مش موجودة"); // هنا الرد اتبعت فعلاً.
```

ب- الـ Status جوه الـ JSON (الخلط اللي بيحصل)

أوقات مبرمجين كتير (وأنا منهم) بنبعت Object فيه كلمة status من عندنا إحنا كـ "بيانات"، ودي مالهاش علاقة بالـ HTTP Status Code:

JavaScript

```
res.status(400).json({
  status: "fail", // داتا JSON دي كلمة من اختراعنا إحنا كـ
  message: "الباسورد غلط"
});
```

هنا الـ **Status Code** هو 400 (وده اللي بيفهمه المتصفح)، والـ **status** اللي جوه الـ JSON دي مجرد "معلومة" إنت باعتها لزميلك بتاع الـ Frontend

---

2. الفرق بين `app.use()` و الميثودز التانية (`app.post` , `app.get` , إلخ)

دي الحتة اللي بتقع من ناس كتير، والفرق بينهم جوهري:

أ- الـ `app.use()` (العمومي):

- **التطابق:** بيبص على "بداية" المسار (Prefix). يعني لو قلت `app.use('/user')` , هيشتغل مع `/user/profile,` `/user/settings` , وحتى لو ناديت عليه بـ `GET` أو `POST` .
- **الوظيفة:** غالباً بيستخدم للـ **Middleware**. حاجات عايزها تتنفذ على مجموعة مسارات مع بعض (زي التأكد إن اليوزر مسجل دخول).
- **المرونة:** ملوش دعوة بنوع الـ HTTP Verb (يعني شغال مع كله).

ب- الـ `app.post` , `app.get()` , إلخ (المتخصص):

- **التطابق:** لازم المسار يكون **بالظبط** (Exact Match). لو قلت `app.get('/user')` , مش هيشتغل مع `user/profile/` .
- **الوظيفة:** تنفيذ أكشن محدد. "هات لي داتا" (`GET`) أو "سيف داتا جديدة" (`POST`).
- **التخصص:** مش هيشتغل غير لو الـ HTTP Method اللي جاية من العميل مطابقة بالمللي.

---

3. الـ Express Router (نظام الأقاليم)

بدل ما تكتب 100 `app.get` مرة، بنستخدم الـ `express.Router()` عشان نعزل الكود.

إزاي بيشتغل داخلياً؟

1. بتعمل ملف جديد (مثلاً `admin.js`).
2. بتعرف فيه الراوتر: `const router = express.Router();` .
3. بترص عليه الطرق بتاعتك: `router.get('/add-product', ...)` .
4. وفي الملف الأساسي (`app.js`)، بتقوله: `app.use('/admin', adminRoutes);` .

---

1. فلترة المسارات (Filtering Paths)

الفكرة ببساطة إنك بدل ما تكرر الكلمة في كل `router` داخلي، بتعمل "فلترة" من بره في الملف الأساسي (`app.js`).

تخيل عندك راوتر للـ **admin** وفيه:

- `/admin/add-product`
- `/admin/products`

الصح إيه؟ في ملف الراوتر (`admin.js`) بتكتب المسارات قصيرة:

وفي ملف الـ `app.js` بتعمل "الفلتر" الكبير:

إيه اللي بيحصل؟ الـ Express بياخد أي طلب بيبدأ بـ `admin/` ويقصه، ويبعت الباقي للراوتر. يعني لو العميل طلب `admin/add-product/` .. الـ Express بيشوف `admin/` في الـ `app.js` فيقوم باعت `add-product/` للراوتر يكمل معاها.

---

2. التشريح العميق لـ `res.sendFile()`

السطر ده هو اللي بيربط "المنطق" (Logic) بـ "الشكل" (UI). تعال نفككه حتة حتة:

```
res.sendFile(path.join(__dirname, "../", "views", "add-product.html"));
```

أ- ليه بنستخدم موديول `path` ؟

إوعى تفكر تكتب المسار كدة: `"views/add-product.html"` . ليه؟

- **اختلاف الأنظمة:** الويندوز بيستخدم `\` في المسارات، واللينكس والماك بيستخدموا `/` . موديول `path` بذكاء بيعرف إنت شغال على إيه وبيحط العلامة الصح.
- **المسار المطلق (Absolute Path):** الـ `res.sendFile` **لازم** تاخد مسار كامل من أول جذور الهارد ديسك لحد الملف، مابتفهمش المسارات النسبية.

ب- إيه هو الـ `__dirname` ؟

ده متغير "عالمي" (Global) في Node.js، وقيمته هي **"عنوان الفولدر اللي فيه الملف اللي شغال دلوقتي"**.

- لو ملفك في `src/routes/admin.js` .. الـ `__dirname` هيوصلك لحد فولدر `routes` .

ج- الـ `path.join` والـ `../` (لعبة السلم والتعبان):

- **`path.join` :** وظيفتها إنها "تلزق" الحتت في بعضها وتطلع مسار سليم.
- **`../` :** دي معناها "اطلع خطوة لبره". لو إنت جوه فولدر الـ `routes` وعايز تروح لـ `views` اللي جنبه، لازم تطلع لبره الأول للمجلد الرئيسي وبعدين تدخل `views` .

---

3. إيه اللي بيحصل "داخلياً" لما السطر ده يتنفذ؟

1. **قراءة الملف:** الـ Node.js بيروح يفتح الهارد ديسك ويقرأ ملف `add-product.html` .

2. **تحديد الـ Content-Type:** الـ Express بيشوف الامتداد `html.` فيقوم ضارب "هيدر" أوتوماتيك للمتصفح يقوله: `Content-Type: text/html`.
3. **الـ Streaming (الصبّ):** الـ Express مش بيقرأ الملف كله في الرامات وبعدين يبعته (عشان لو الملف كبير السيرفر مايقعش)، هو بيفتحه كـ **Stream** ويصبه للمتصفح حتة حتة بسرعة جداً.
4. **القفل:** أول ما الملف يخلص، الـ Express بيقفل الطلب أوتوماتيك.

---

1. تشريح السطر: `require.main.filename`

- `node` بيمثل الملف اللي "بدأ" الليلة كلها. يعني لو إنت شغلت السيرفر بـ `app.js`. هو ملف الـ `require.main` يبقى .. `app.js`
- `require.main` : ده كائن (Object) بيمثل الملف اللي "بدأ" الليلة كلها. يعني لو إنت شغلت السيرفر بـ `app.js`. هو ملف الـ `require.main` يبقى .. `app.js`
- `.filename` : المسار المطلق "ده بيجيب لك (Absolute Path) للملف ده بالكامل، شامل اسمه وامتداده.
  - *مثلاً:* `C:\Users\Expert\Desktop\my-project\app.js`

2. وظيفة `path.dirname()`

الدالة دي من موديول الـ `path` بتعمل "قص" لاسم الملف وبتاخد العنوان اللي قبله بس.

- *النتيجة:* بدل ما كان معاك المسار لحد `app.js` .. بقى معاك المسار لحد فولدر `my-project` بس. **وده هو الـ Root بتاعك!**

---

3. ليه `require.main` بدل `process.mainModule` ؟

- `process.mainModule` : دي الطريقة القديمة (Legacy). "اعتبرها "بيئة قديمة Node.js هي لسه شغالة بس الـ (Deprecated) وممكن تتشال في المستقبل.
- `require.main` : الحديثة (زي اللي إنت شغال بيها في Node هي الطريقة الرسمية والحديثة اللي بنستخدمها من أول نسخ 2026). هي أضمن وأدق في تحديد مين "رأس الحية" (الملف الأساسي).

---

4. إيه الفائدة العظيمة للسطر ده؟ (الزتونة)

تخيل إنك جوه ملف `src/routes/admin/products/controller.js` وعايز تبعت ملف HTML موجود في `views/index.html`.

- **من غير السطر ده:** هتضطر تقعد تعد: `views/index.html/../../../..` (لو نقلت الملف في فولدر تاني، الكود هيضرب!).
- **بالسطر ده:** إنت معاك "ثابت" (Constant) بيوصلك لأول المشروع علطول.

طريقة الاستخدام الاحترافية:

1. تعمل ملف اسمه `util/path.js` :

JavaScript

```
const path = require('path');
module.exports = path.dirname(require.main.filename);
```

2. تستخدمه في أي حتة في المشروع كدة:

JavaScript

```
const rootDir = require('../util/path'); // بتنادي البوصلة بتاعتك
const path = require('path');

res.sendFile(path.join(rootDir, 'views', 'add-product.html'));
```

شايف الشياكة؟ مفيش `../` تانية أبداً، الكود بقى "نظيف" ومفهوم.

---

1. يعني إيه "Static Files"؟

هي الملفات اللي السيرفر مابيعملش فيها أي مجهود أو تغيير. بياخدها من الهارد ديسك يرميها للمتصفح زي ما هي. (زي صورة لوجو الموقع، أو ملف تنسيق الألوان CSS).

2. وظيفة `express.static` (الرد الآلي)

من غير السطر ده، لو الزبون طلب `localhost:3000/css/main.css` .. السيرفر هيقوله: "معرفش، الرابط ده مش عندي في الراوتر".

الـ `express.static` بيعمل إيه؟ بيقول للـ Express: "يا ريس، لو حد طلب أي ملف وإنت مالقيتوش في الراوتر بتاعك، روح بص بصه سريعة جوه فولدر `public` . لو لقيت الملف هناك، ابعتهوله علطول من غير ما توجع دماغي".

---

3. تشريح السطر: (التكات الفنية)

`app.use(express.static(path.join(__dirname, "public")));`

- عشان `public` إحنا خلاص عرفنا إن دي بتجيب المسار المطلق لفولدر : `path.join(__dirname, "public")` السيرفر ما يتوهش.
- **الإتاحة:** أول ما بتعمل السطر ده، أي حاجة جوه فولدر `public` بتبقى **"مشاع" (Publicly Accessible)**.

**** ⚠️ ملاحظة**

لما الزبون بيطلب الملف، **مابيكتبش** كلمة `public` في الرابط!

- **صح:** لو الملف مكانه `public/css/main.css` .. الزبون بيطلبه كدة: `/css/main.css` .
- **غلط:** إنه يكتب `/public/css/main.css` .
- **ليه؟** لأن الـ Express بيعتبر إن فولدر `public` هو "الجذر" (Root) بتاع الملفات دي، وكأنه مش موجود في العنوان.

---

4. إيه اللي بيحصل داخلياً (Inside the Engine)؟

1. الطلب بيوصل للسيرفر.
2. الـ Express بيمشي في الـ Middleware بالترتيب.
3. بيخبط في `express.static` .
4. المفتش بيشوف الرابط.. "هل فيه ملف في `public` بالاسم ده؟"

- **لو لقى:** بيحدد الـ `Content-Type` أوتوماتيك (مثلاً `text/css` أو `image/png`) ويبعت الملف ويقفل الطلب.
- **لو مالقاش:** بيدوس `next()` بمنتهى الهدوء عشان الطلب يكمل ويروح للراوتر اللي بعده (يمكن يكون `404` مثلاً).